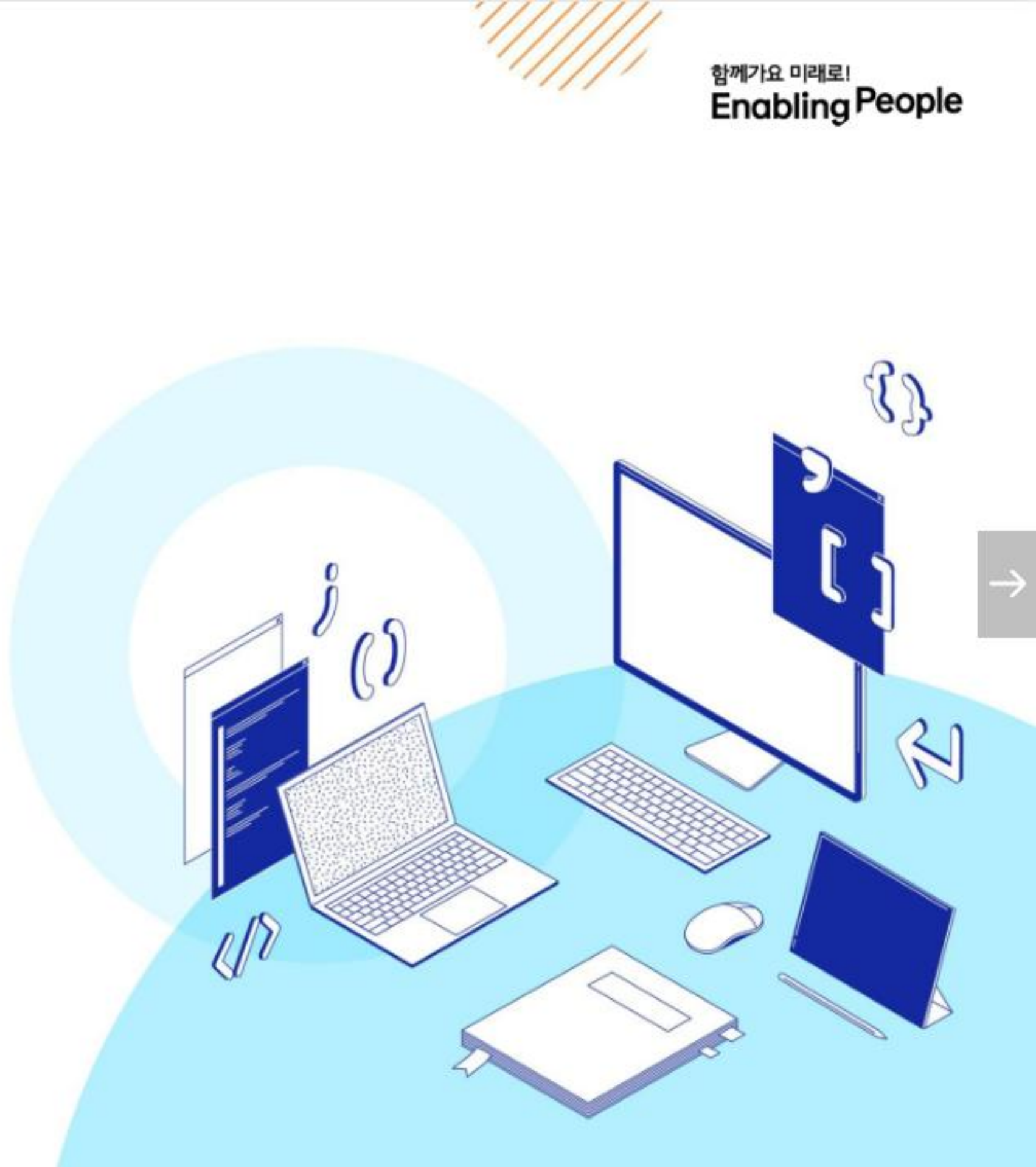


SSAFY

Javascript

Controlling event

Python



목 차

이벤트

- event
- event object
- event handler

버블링

- 캡처링과 버블링
- 버블링의 필요성

목 차

event handler 활용

- 이벤트 기본 동작 취소하기

참고

- addEventListener와 화살표 함수 관계

웹 페이지를 하나의 '카페'라고 상상해봅시다. 손님 주문은 어떻게 처리될까요?

- 카페에서 손님이 주문하는 과정을 살펴봅시다.

1. 손님이 키오스크에서 하는 '주문'(클릭, 입력 등)이 바로 []입니다.
2. 이 주문을 받는 키오스크가 []입니다.
3. 주문에 맞춰 커피를 만드는 바리스타의 행동이 []입니다.

- 웹 페이지에서 다양한 클릭, 입력을 받아 처리하는 방식을 학습합니다.
- 학습 후 카페에서 손님이 주문하는 과정과 웹에서의 이벤트 처리 방식을 비교하면서 학습을 마무리합니다.

학습 목표

- ✓ `addEventListener`를 사용해 이벤트 핸들러를 등록할 수 있다.
- ✓ 이벤트 핸들러의 `event` 객체를 활용해 정보를 얻을 수 있다.
- ✓ 이벤트 버블링의 개념과 전파 과정을 설명할 수 있다.
- ✓ `event.target`과 `event.currentTarget`의 차이점을 이해한다.
- ✓ 이벤트 버블링을 활용하여 상위 요소에 이벤트를 위임한다.
- ✓ `event.preventDefault()`로 요소의 기본 동작을 취소한다.
- ✓ DOM 조작과 이벤트를 결합하여 동적인 웹을 구현한다.

이벤트

일상속의 이벤트

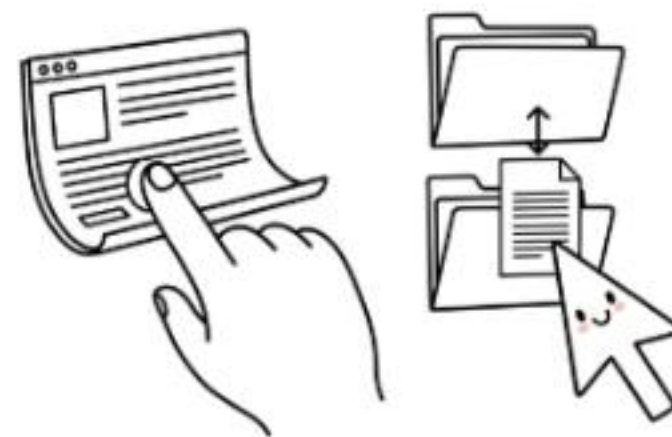
- 컴퓨터 키보드를 눌러 텍스트를 입력하는 것
- 전화벨이 울려 전화가 왔음을 알리는 것
- 손을 흔들어 인사하는 것
- 전화기의 버튼을 눌러서 통화를 시작하는 것
- 리모컨을 사용하여 채널을 변경하는 것



〈그림1_Javascript_controlling event_일상속의 이벤트〉

웹에서의 이벤트

- 화면을 스크롤하는 것
- 버튼을 클릭했을 때 팝업 창이 출력되는 것
- 마우스 커서의 위치에 따라 드래그 앤 드롭하는 것
- 사용자의 키보드 입력 값에 따라 새로운 요소를 생성하는 것



➤ 웹에서의 거의 **모든 상호작용은 이벤트와 함께 함**

<그림2_Javascript_controlling event_웹에서의 이벤트>

event

event

웹 페이지 상에서
'무언가 일어났다는 신호 또는 사건'

JS 이벤트는 '영화 서비스'와 같습니다.

우리가 보고 싶은 영화 포스터에 마우스를 올리면(이벤트), 시스템은 마우스의 움직임을 감지(리스너)하고, 작은 미리보기 영상이나 설명을 보여주는(핸들러) 반응을 보입니다.

마우스 움직임 하나하나가 이벤트가 될 수 있고, 이러한 다양한 클릭, 입력, 변경하는 행위가 이벤트에 해당합니다.

DOM 요소와 이벤트

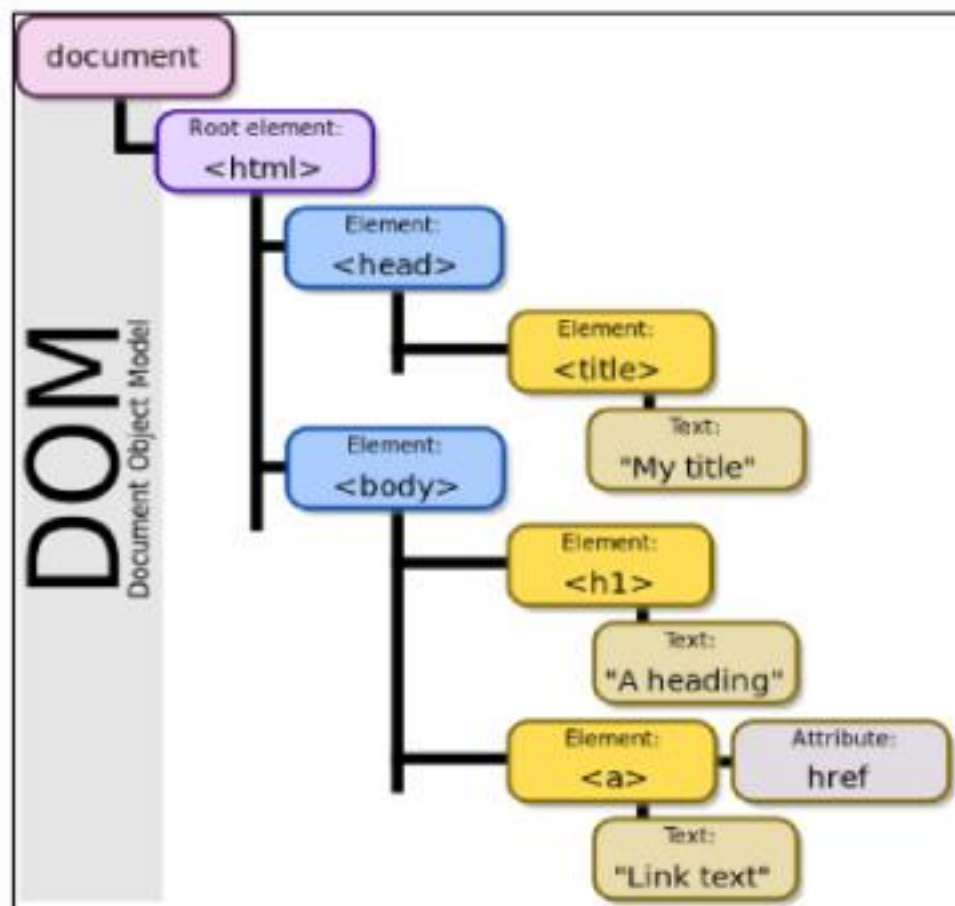
- 모든 DOM 요소는 다양한 형태의 이벤트를 발생시킬 수 있음
- 예) button 을 클릭하면 click 이벤트, input 값 변경 시 input 이벤트, ...



DOM 요소

HTML 문서의 각 태그를 하나의 객체로 변환한 것

<개념1_Javascript_controlling event_DOM 요소>

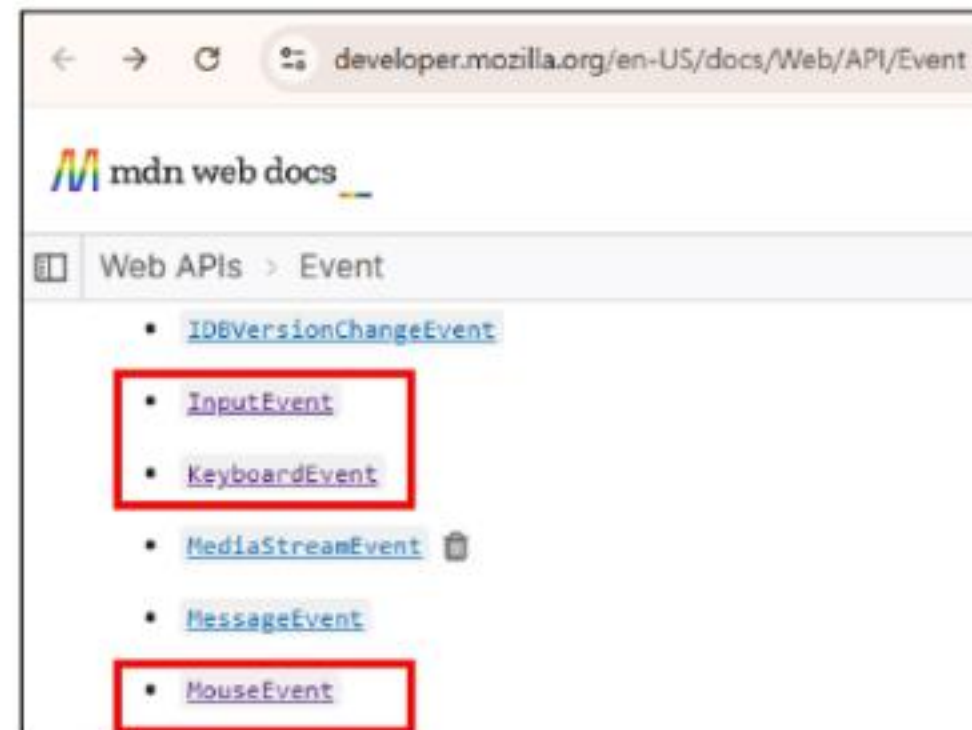


<그림3_Javascript_controlling event_DOM 예시>

event object

event object

- DOM에서 이벤트가 발생하면, 브라우저는 해당 이벤트에 관한 정보를 담은 'event object'를 자동으로 생성
- 이벤트 종류
 - mouse
 - input
 - keyboard
 - ...



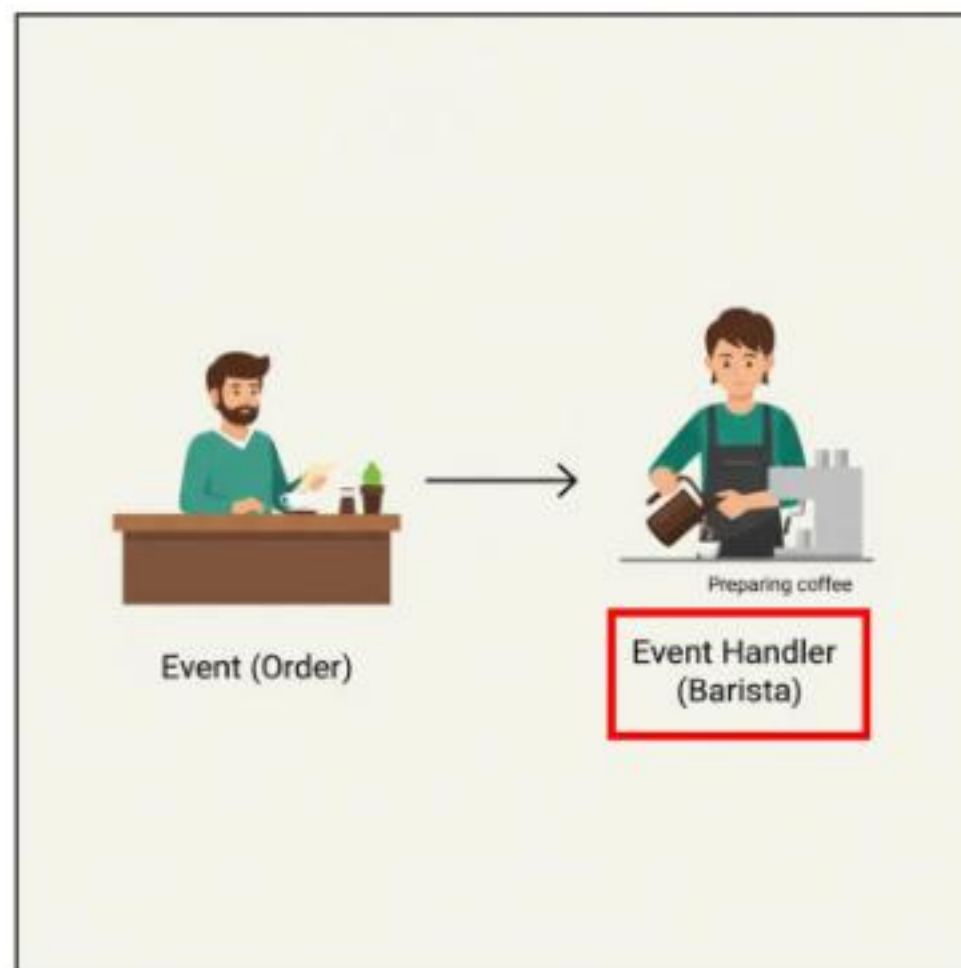
〈그림4_Javascript_Controlling event_event 종류〉

TIP

- 이벤트 객체는 이벤트 발생 순간의 상황(어떤 요소에서 이벤트가 발생했는지, 마우스 좌표는 어디인지, 눌린 키는 무엇인지 등)과 관련된 상세 정보를 담고 있습니다.
- 이를 통해 이벤트와 관련된 구체적인 정보를 참조할 수 있습니다.

DOM 요소에서 event가 발생하면,

해당 event는 연결된 이벤트 처리기(event handler)에 의해 처리 됨



<그림5_Javascript_Controlling event_event handler 예시>

event handler

event handler

event handler

특정 이벤트가 발생했을 때 실행되는
(콜백)함수

JS 이벤트는 '영화 서비스'와 같습니다.

우리가 보고 싶은 영화 포스터에 마우스를 올리면(이벤트), 시스템은 마우스의 움직임을 감지(리스너)하고,
작은 미리보기 영상이나 설명을 보여주는(핸들러) 반응을 보입니다.

마우스 움직임 하나하나가 이벤트가 될 수 있고, 이러한 다양한 클릭, 입력, 변경하는 행위가 이벤트에 해당합니다.

※ 보통 `addEventListener`를 통해 DOM 요소에 event handle를 등록합니다.

.addEventListener()

.addEventListener()

특정 DOM 요소에, 지정한 이벤트가 발생했을 때 실행할 이벤트 핸들러를 등록하는 메서드

JS 이벤트는 '영화 서비스'와 같습니다.

우리가 보고 싶은 영화 포스터에 마우스를 올리면(이벤트), 시스템은 마우스의 움직임을 감지(리스너)하고, 작은 미리보기 영상이나 설명을 보여주는(핸들러) 반응을 보입니다.

마우스 움직임 하나하나가 이벤트가 될 수 있고, 이러한 다양한 클릭, 입력, 변경하는 행위가 이벤트에 해당합니다.

※ 이벤트 핸들러(콜백 함수)를 DOM 요소에 연결하는 역할을 담당합니다

.addEventListener() 예시

- handleClick 함수가 이벤트 핸들러이며,
button.addEventListener()는 그 핸들러를 click 이벤트에 연결해주는 역할

```
const button = document.querySelector('button')

// 이벤트 핸들러
const handleClick = function () {
  window.alert('버튼이 클릭 되었습니다!')
}

// addEventListener 메서드를 이용해 버튼에 이벤트 핸들러를 등록
button.addEventListener('click', handleClick)
```



이벤트 핸들러
특정 이벤트가 발생했을 때
실행되는 (콜백)함수

<개념2_Javascript_Controlling event_이벤트 핸들러>



.addEventListener()
특정 DOM 요소에, 지정한
이벤트가 발생했을 때 실행할
이벤트 핸들러를 등록하는
메서드

<개념3_Javascript_Controlling event_.addEventListener()>

이벤트 등록(addEventListener)

- **DOM 요소**: HTML 문서의 각 태그를 하나의 객체로 변환한 것
- **수신할 이벤트**: 무언가 일어났다는 신호 또는 사건
- **핸들러**: 특정 이벤트가 발생했을 때 실행되는 (콜백)함수

EventTarget.addEventListener(**type**, **handler**)

DOM 요소

리스너

수신할 이벤트

핸들러

<그림6_Javascript_Controlling event_addEventListener 설명>

addEventListener 구조

- 메서드 구문
 - `.addEventListener(type, handler)`
- type
 - 수신할 이벤트 유형
 - 문자열로 작성 (ex. 'click', 'mouseover' 등)
- handler
 - 이벤트 발생 시 호출되는 콜백 함수
 - 자동으로 event 객체를 첫번째 매개변수로 받음
 - 반환 값 없음

```
element.addEventListener('click', function (event) {  
    // 이벤트 처리 로직  
})
```

이벤트 객체 전달

- 이벤트 발생 시, 이벤트 객체는 자동으로 이벤트 핸들러 함수에 인자로 전달됨
- 핸들러 함수는 이 인자를 통해 이벤트에 대한 상세 정보(이벤트 발생 요소, 이벤트 타입, 추가 데이터 등)에 접근하고 적절한 동작을 수행

```
<button id="btn">버튼</button>
```

```
// 1. 버튼 요소 선택
const btn = document.querySelector('#btn')

// 2. 이벤트 핸들러
const detectClick = function (event) {
  console.log(event) // PointerEvent
  console.log(event.type) // click
}

// 3. 버튼에 이벤트 핸들러를 등록
btn.addEventListener('click', detectClick)
```


이벤트 핸들러에서의 this

- 일반 함수를 핸들러로 사용 시, this는 이벤트 리스너가 연결된 요소를 가리킴
(event 객체의 currentTarget 속성 값과 동일)

<button id="btn">버튼</button>

```
// 1. 버튼 선택
const btn = document.querySelector('#btn')

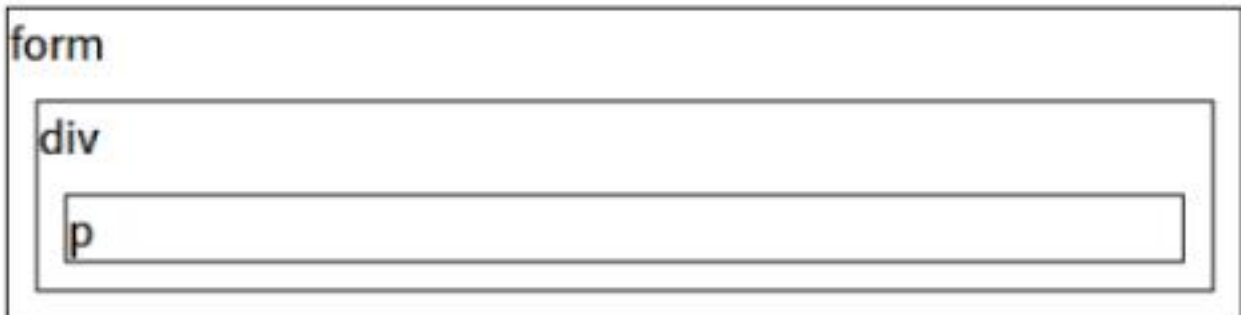
// 2. 이벤트 핸들러
const detectClick = function (event) {
  console.log(event.currentTarget) // <button id="btn">버튼</button>
  console.log(this) // <button id="btn">버튼</button>
}

// 3. 버튼에 이벤트 핸들러를 등록
btn.addEventListener('click', detectClick)
```


버블링

버블링 개요 (1/2)

- form > div > p 형태의 중첩된 구조에 각각 이벤트 핸들러가 연결되어 있을 때,
만약 <p> 요소를 클릭하면 어떻게 될까?



<그림7_Javascript_Controlling event_버블링 예시>

```
<form id="form">
  form
  <div id="div">
    div
    <p id="p">p</p>
  </div>
</form>
```

```
const formElement = document.querySelector('#form')
const divElement = document.querySelector('#div')
const pElement = document.querySelector('#p')
```

```
const clickHandler1 = function (event) {
  console.log('form이 클릭되었습니다.')
}
```

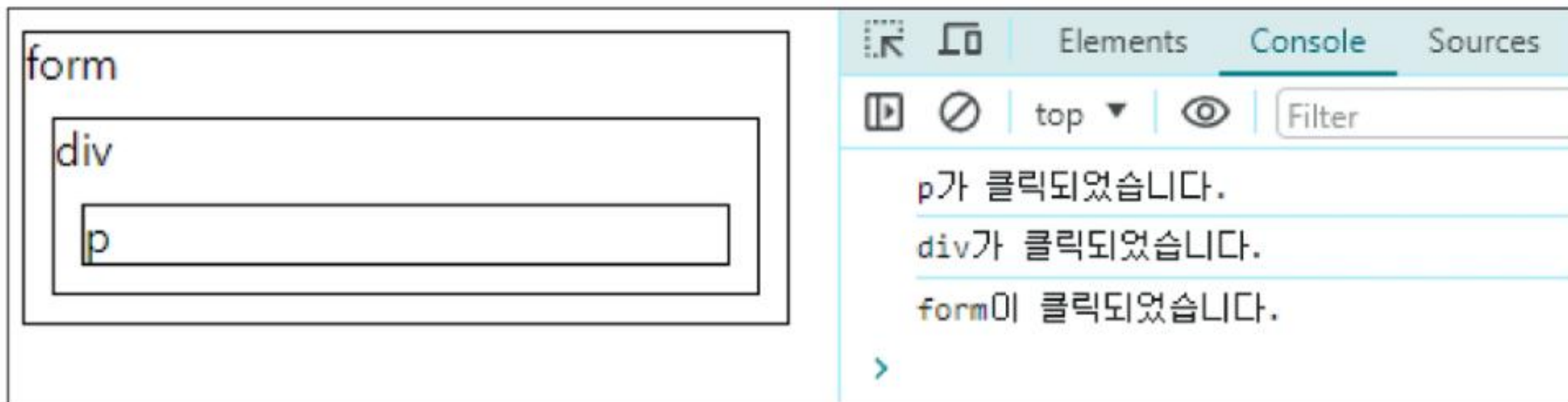
```
const clickHandler2 = function (event) {
  console.log('div가 클릭되었습니다.')
}
```

```
const clickHandler3 = function (event) {
  console.log('p가 클릭되었습니다.')
}
```

```
formElement.addEventListener('click', clickHandler1)
divElement.addEventListener('click', clickHandler2)
pElement.addEventListener('click', clickHandler3)
```

버블링 개요 (2/2)

- <p> 요소만 클릭했는데도 불구하고 모든 핸들러가 동작함
- 왜 p만을 클릭했는데 부모 요소 <div>와 <form>에 할당된 핸들러까지 동작할까?

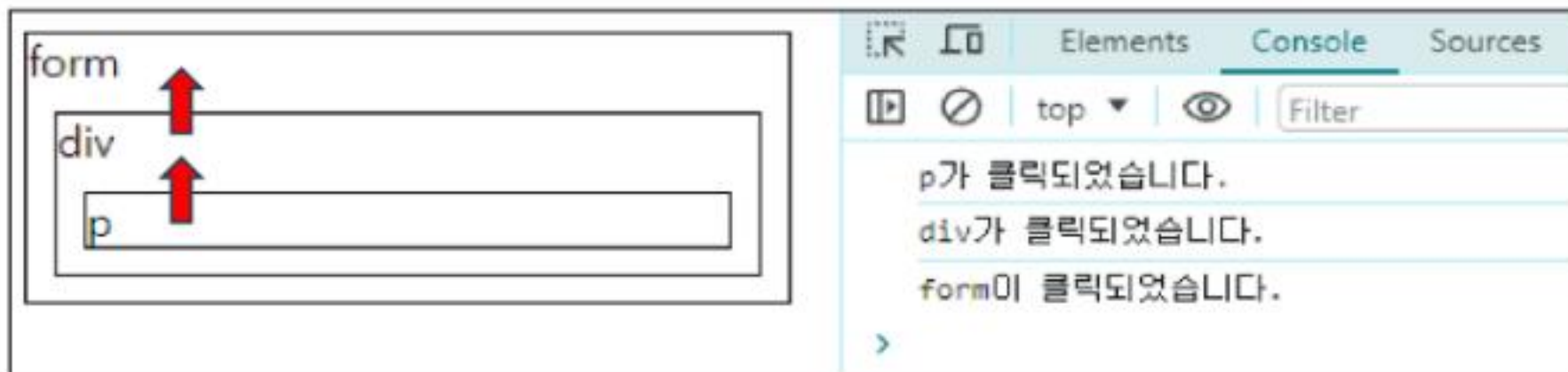


The diagram illustrates event bubbling in a web browser. On the left, a DOM tree is shown with three nested boxes: an outer box labeled 'form', a middle box labeled 'div' inside 'form', and an inner box labeled 'p' inside 'div'. On the right, the browser's developer console is open, showing three log messages in Korean: 'p가 클릭되었습니다.' (p was clicked.), 'div가 클릭되었습니다.' (div was clicked.), and 'form이 클릭되었습니다.' (form was clicked.). The console tabs are 'Elements', 'Console', and 'Sources', with 'Console' selected. The log messages are ordered from top to bottom as they appear in the console.

<그림8_Javascript_Controlling event_버블링 예시 로그>

버블링 (Bubbling)

- 한 요소에 이벤트가 발생하면, 해당 요소의 핸들러가 동작한 후 이어서 부모 요소의 핸들러가 동작하는 현상
- 가장 최상단의 조상 요소(document)를 만날 때까지 이 과정이 반복되면서 요소 각각에 할당된 핸들러가 동작



<그림9_Javascript_Controlling event_버블링 예시 과정>

- 최하위의 <p> 요소를 클릭하면 p → div → form 순서로 3개의 이벤트 핸들러가 모두 순차적으로 동작했던 것

TIP

- 이벤트가 제일 깊은 곳에 있는 요소에서 시작해 부모 요소를 거슬러 올라가며 발생하는 것이 마치 물속 거품과 닮았기 때문에 버블링이라고 해요

이벤트가 정확히 어디서 발생했는지 접근할 수 있는 방법

1. event.currentTarget

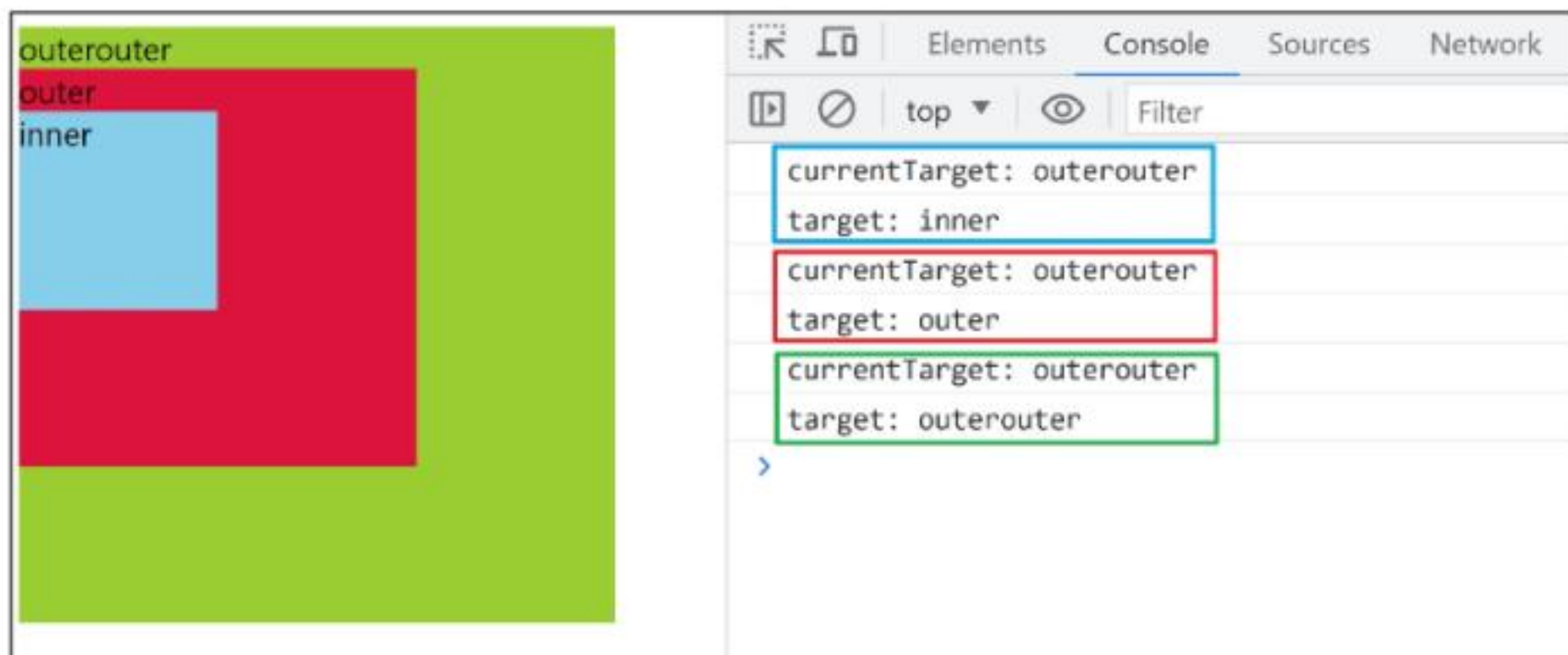
- '현재' 요소
- 항상 이벤트 핸들러가 연결된 요소만을 참조하는 속성
- 'this' 와 같음

2. event.target

- 이벤트가 발생한 가장 안쪽의 요소(target)를 참조하는 속성
- 실제 이벤트가 시작된 요소
- 버블링이 진행 되어도 변하지 않음

'target' & 'currentTarget' 예시 (1/3)

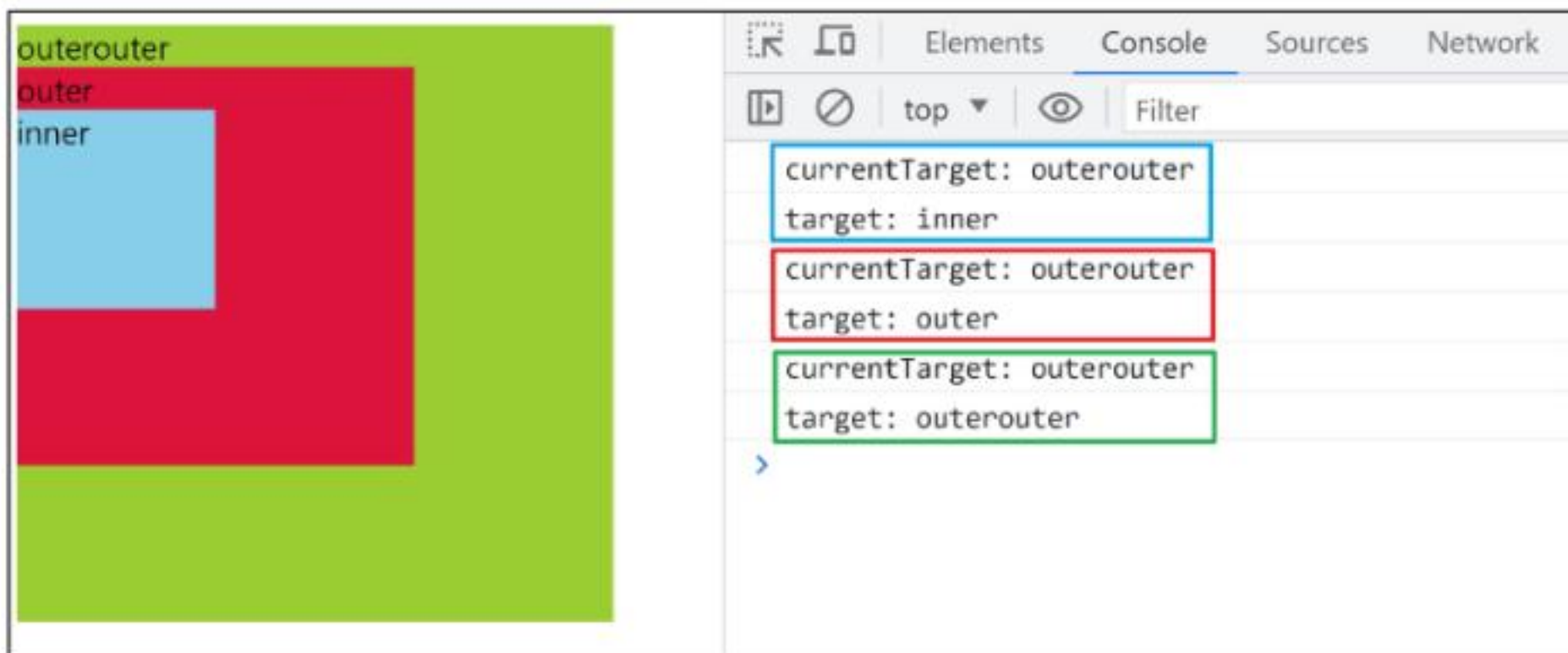
- 세 요소 중 가장 최상위 요소인 **outerouter** 요소에만 핸들러가 연결
- 각 요소를 클릭 했을 때 event의 target과 currentTarget의 차이 비교



<그림10_Javascript_Controlling event_버블링 예시2>

'target' & 'currentTarget' 예시 (2/3)

- 핸들러는 outerouter에만 연결되어 있지만, 하위 요소 outer와 inner를 클릭해도 해당 핸들러가 동작함
- 클릭 이벤트가 어디서 발생했든 상관없이 outerouter까지 이벤트가 버블링 되어 핸들러를 실행시키기 때문



<그림10_Javascript_Controlling event_버블링 예시2>



currentTarget

핸들러가 연결된 outerouter
요소만을 가리킴

<개념4_Javascript_Controlling event_currentTarget>

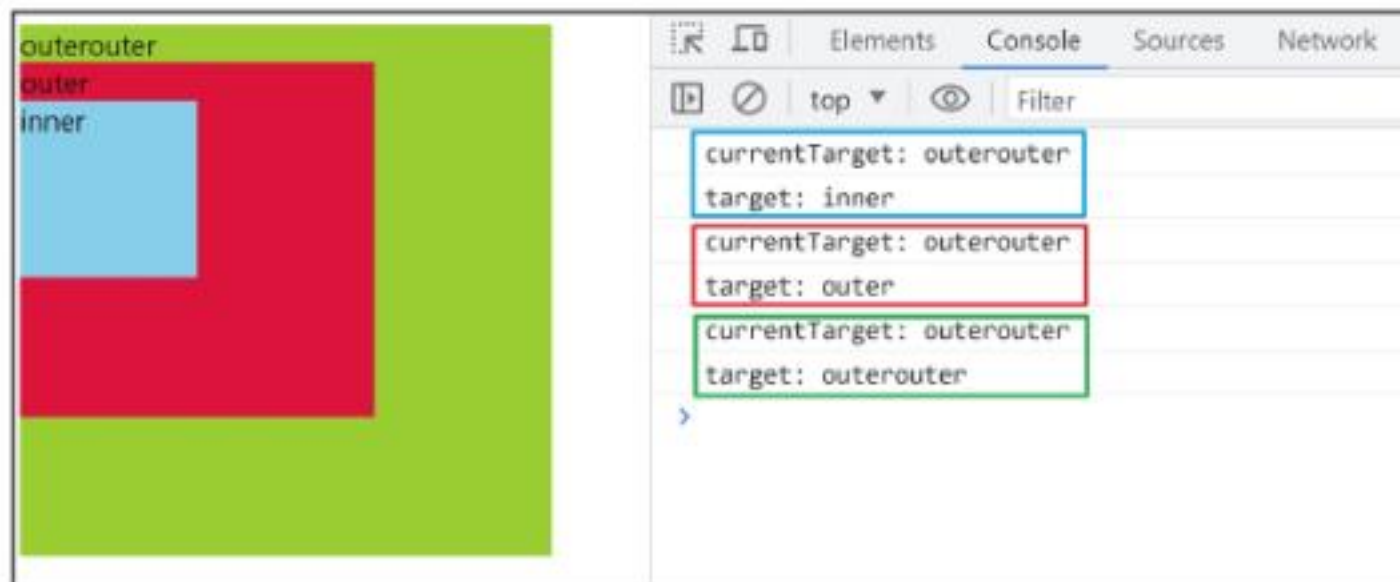


target

실제 이벤트가 발생하는 요소를
가리킴

<개념5_Javascript_Controlling event_target>

'target' & 'currentTarget' 예시 (3/3)



<그림10_Javascript_Controlling event_버블링 예시2>

```
#outer {
  width: 200px;
  height: 200px;
  background-color: crimson; }
#inner {
  width: 100px;
  height: 100px;
  background-color: skyblue; }
#outerouter {
  width: 300px;
  height: 300px;
  background-color: yellowgreen; }
```

```
<div id="outerouter">
  outerouter
  <div id="outer">
    outer
    <div id="inner">inner</div>
  </div>
</div>
<script>
  const outerOuterElement =
document.querySelector('#outerouter')

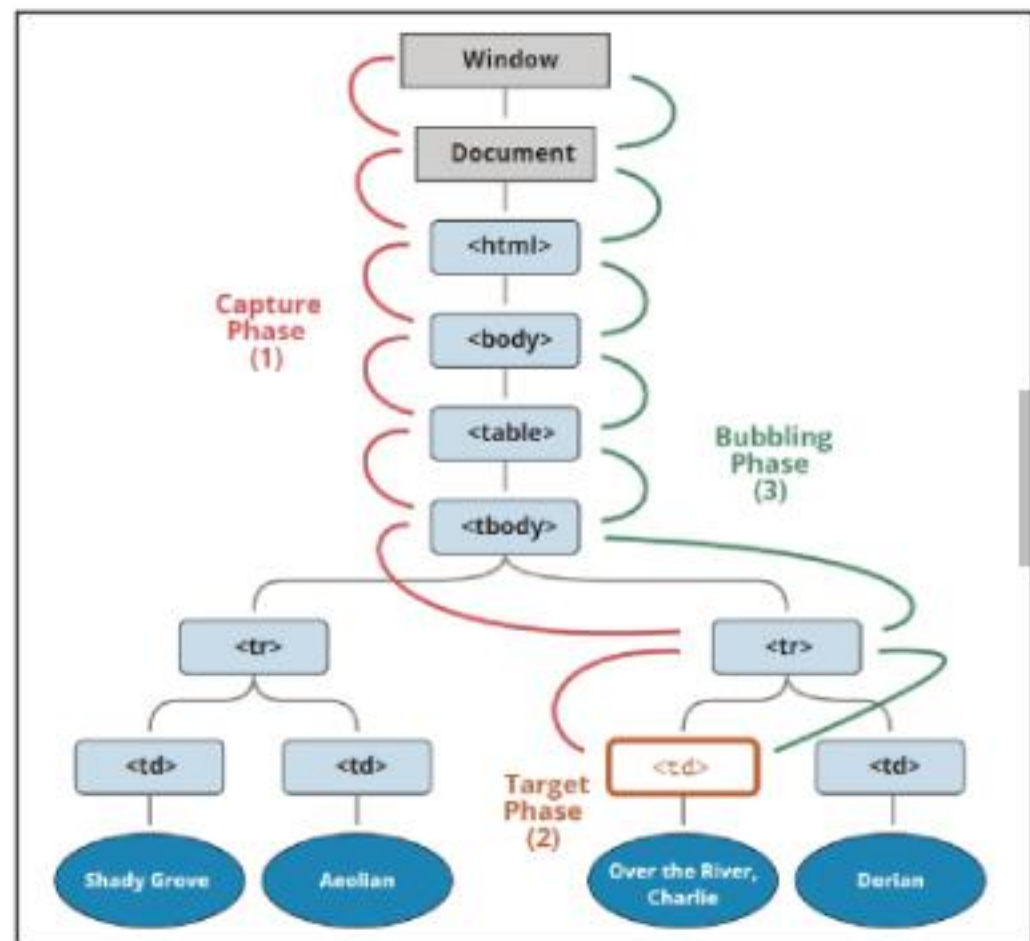
  const clickHandler = function (event) {
    console.log('currentTarget:', event.currentTarget.id)
    console.log('target:', event.target.id)
  }

  outerOuterElement.addEventListener('click', clickHandler)
</script>
```


캡처링과 버블링

캡처링(capturing)

- 이벤트가 최상위 조상에서 타겟 요소까지 하위로 전파되는 단계 (버블링과 반대)
- table 의 하위 요소 td를 클릭하면,
이벤트는 먼저 **최상위 요소부터 아래로 전파**됨 (캡처링)
- 실제 이벤트가 발생한 지점(event.target)에서 실행된 후
다시 위로 전파 (버블링)



<그림11_Javascript_Controlling event_캡처링>

TIP

- 캡처링은 실제 개발자가 다루는 경우가 거의 없으므로 버블링을 집중해서 학습하는 것이 좋습니다.

버블링의 필요성

버블링이 필요한 이유 (1/2)

- 만약 다음과 같이 각자 다른 동작을 수행하는 버튼이 여러 개 있다고 가정하면, 각 버튼마다 서로 다른 이벤트 핸들러를 등록해야 할까?

➤ 각 버튼의 공통 조상인 div 요소에 이벤트 핸들러 단 하나만 할당하기

```
<div>  
  <button></button>  
  <button></button>  
  ...  
  <button></button>  
  <button></button>  
</div>
```


버블링이 필요한 이유 (2/2)

- 요소들의 공통 조상에 이벤트 핸들러를 **하나만 등록**하면, 여러 자식 요소에서 발생하는 이벤트를 한곳에서 효율적으로 다룰 수 있음
- 공통 조상(div)에 할당한 핸들러에서 event.target을 이용하면 실제 어떤 버튼에서 이벤트가 발생했는지 알 수 있기 때문

```
<div>  
  <button></button>  
  <button></button>  
  ...  
  <button></button>  
  <button></button>  
</div>
```

// 예시

```
const divTag = document.querySelector('div')  
  
divTag.addEventListener('click', function (event) {  
  console.log(event.target)  
})
```

event handler 활용

event handler 활용 실습 목록

1. 버튼을 클릭하면 숫자를 1씩 증가해서 출력하기
2. 사용자 입력 값을 실시간으로 출력하기
3. 사용자 입력 값을 실시간으로 출력하기 + 버튼을 클릭하면 출력된 값의 CSS 스타일을 변경하기
4. todo 프로그램 구현
5. 로또 번호 생성기 구현

1. click 이벤트 실습

- 버튼을 클릭하면 숫자를 1씩 증가



〈그림12_Javascript_Controlling event_click 이벤트 실습〉

```
<!-- click-event.html -->
```

```
<button id="btn">버튼</button>
<p>클릭횟수 : <span
id="counter">0</span></p>
```

```
// 1. 초기값 할당
let counterNumber = 0

// 2. 버튼 요소 선택
const btn = document.querySelector('#btn')

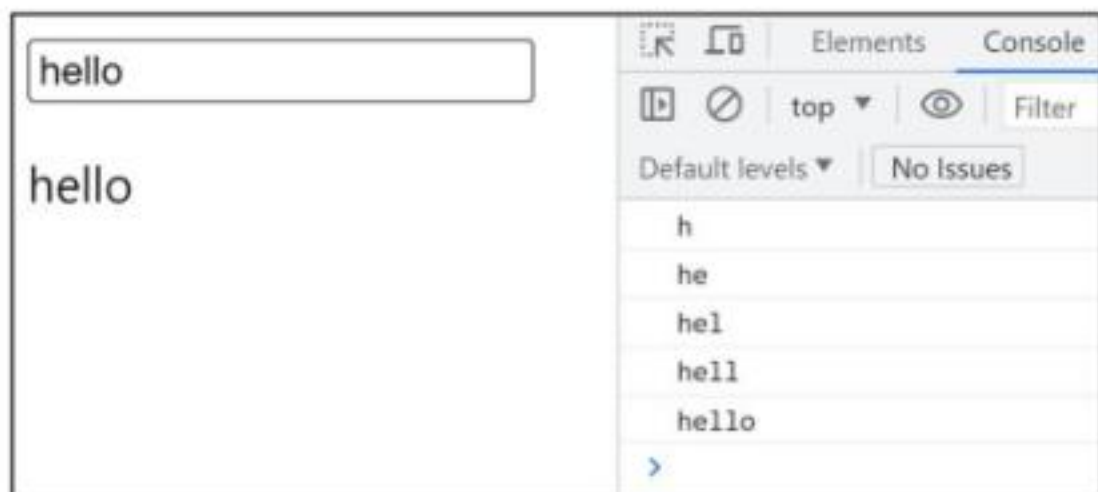
// 3. 콜백 함수 (버튼에 클릭 이벤트가 발생할때마다 실행할 코드)
const clickHandler = function () {
  // 3.1 초기값 += 1
  counterNumber += 1

  // 3.2 p 요소를 선택
  const spanTag = document.querySelector('#counter')
  // 3.3 p 요소의 콘텐츠를 1 증가한 초기값으로 설정
  spanTag.textContent = counterNumber
}

// 4. 버튼에 이벤트 핸들러 등록 (클릭 이벤트)
btn.addEventListener('click', clickHandler)
```


2. input 이벤트 실습

- 사용자의 입력 값을 실시간으로 출력하기



<그림13_Javascript_Controlling event_input 이벤트 실습>

```
<!-- input-event.html -->
```

```
<input type="text" id="text-input">
<p></p>
```

```
// 1. input 요소 선택
const inputTag = document.querySelector('#text-input')

// 2. p 요소 선택
const pTag = document.querySelector('p')

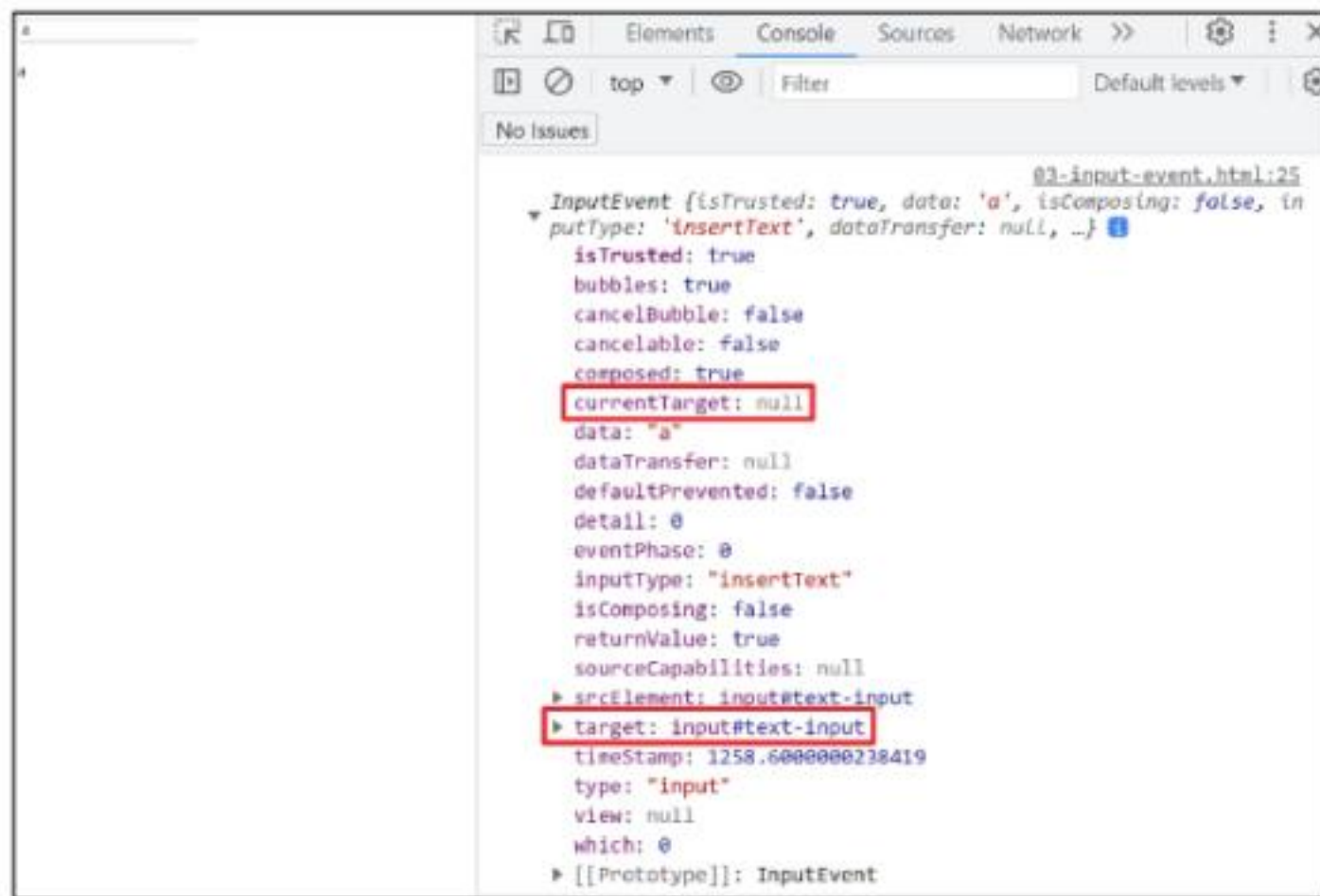
// 3. 콜백 함수
// (input 요소에 input 이벤트가 발생할때마다 실행할 코드)
const inputHandler = function (event) {
  // 3.1 작성하는 데이터가 어디에 누적되고 있는지 찾기
  // console.log(event.currentTarget)
  console.log(event.currentTarget.value)

  // 3.2 p요소의 콘텐츠에 작성하는 데이터를 추가
  pTag.textContent = event.currentTarget.value
}

// 4. input 요소에 이벤트 핸들러 등록 (input 이벤트)
inputTag.addEventListener('input', inputHandler)
```

'currentTarget' 주의사항

- console.log()로 event 객체를 출력할 경우, currentTarget 키의 값은 null을 가짐
- currentTarget은 이벤트가 처리되는 동안에만 사용할 수 있기 때문
- 대신 console.log(event.currentTarget)로 확인 가능
- currentTarget 이후의 속성 값들은
 'target'을 참고해서 사용하기



<그림14_Javascript_Controlling event_currentTarget 주의사항>

3. click & input 이벤트 실습

- 입력 값을 실시간으로 출력하고,
클릭 버튼을 클릭 시 출력 값의 CSS 변경하기



hello

<그림15_Javascript_Controlling event_click input 이벤트 실습>

```
<!-- click-input-event.html -->
```

```
<head>
  <style>
    .blue {
      color: blue;
    }
  </style>
</head>

<body>
  <h1></h1>
  <button id="btn">클릭</button>
  <input type="text" id="text-input">
</body>
```

```
// input 구현
const inputTag = document.querySelector('#text-input')
const h1Tag = document.querySelector('h1')

const inputHandler = function (event) {
  h1Tag.textContent = event.currentTarget.value
}

inputTag.addEventListener('input', inputHandler)

// click 구현
const btn = document.querySelector('#btn')

const clickHandler = function () {
  // 1. add 방법
  h1Tag.classList.add('blue')

  // 2. toggle 방법
  h1Tag.classList.toggle('blue')

  // 3. if 방법
  if (h1Tag.classList.value) {
    h1Tag.classList.remove('blue')
  } else {
    h1Tag.classList.add('blue')
  }
}

btn.addEventListener('click', clickHandler)
```

4. todo 실습 (1/2)

- 입력 후 '+' 버튼 클릭 시, 실시간으로 리스트에 출력

- 파이썬 공부
- 알고리즘 공부
- 저녁 먹기

<그림16_Javascript_Controlling event_todo 실습>

```
<!-- todo.html -->

<input type="text" class="input-text">
<button id="btn">+</button>
<ul></ul>
```

```
// 1. 필요한 요소 선택
const inputTag = document.querySelector('.input-text')
const btn = document.querySelector('#btn')
const ulTag = document.querySelector('ul')

const addTodo = function (event) {
  // 2.1 사용자 입력 데이터 저장
  const inputData = inputTag.value

  // 2.2 데이터를 저장할 li 요소를 생성
  const liTag = document.createElement('li')

  // 2.3 li 요소 콘텐츠에 데이터 입력
  liTag.textContent = inputData

  // 2.4 li 요소를 부모 ul 요소의 자식 요소로 추가
  ulTag.appendChild(liTag)

  // 2.5 todo 추가 후 input의 입력 데이터는 초기화
  inputTag.value = ''
}
```


4. todo 실습 (2/2)

• 추가 기능 구현

1. 빈 문자열 입력 방지
2. 입력이 없을 경우 경고 대화상자를 띄움

```
<!-- todo.html -->

const addTodo = function (event) {
  const inputData = inputTag.value
  if (inputData.trim()) {
    const liTag = document.createElement('li')
    liTag.textContent = inputData
    ulTag.appendChild(liTag)
    inputTag.value = ''
  } else {
    alert('할 일을 입력하세요..')
  }
}
```

5. 로또 번호 생성기 실습

- '행운 번호 받기' 버튼 클릭 시,
6자리 랜덤 숫자가 실시간으로 리스트에 출력

로또 추천 번호

행운 번호 받기

- 8
- 30
- 5
- 11
- 14
- 4
- 5
- 40
- 24
- 25
- 27
- 26

<그림17 Javascript Controlling event_로또 번호 생성기 실습>

```
<!-- lottery.html -->
<h1>로또 추천 번호</h1>
<button id="btn">행운 번호 받기</button>
<div></div>
```

```
<script src="https://cdn.jsdelivr.net/npm/lodash@4.17.21/lodash.min.js"></script>
<script>
  // 1. 필요한 요소 선택
  const h1Tag = document.querySelector('h1')
  const btn = document.querySelector('#btn')
  const divTag = document.querySelector('div')

  // 2. 로또 번호를 생성하는 함수
  const getLottery = function (event) {
    const numbers = _.range(1, 46) // 2.1 1부터 45까지의 값이 필요
    // 2.2 45개의 요소가 있는 배열에서 6개 번호 추출
    const sixNumbers = _.sampleSize(numbers, 6)
    // 2.5 6개의 li 요소를 담은 ul 요소 생성
    const ulTag = document.createElement('ul')

    // 2.3 추출한 번호 배열을 "반복"하면서 li 요소를 생성
    sixNumbers.forEach((number) => {
      const liTag = document.createElement('li') // 2.4 번호를 담은 li 요소 생성 후 입력

      liTag.textContent = number
      ulTag.appendChild(liTag) // 2.6 만들어진 li를 ul 요소에 추가
    })
    // 2.7 완성한 ul 요소를 div 요소에 추가
    divTag.appendChild(ulTag)

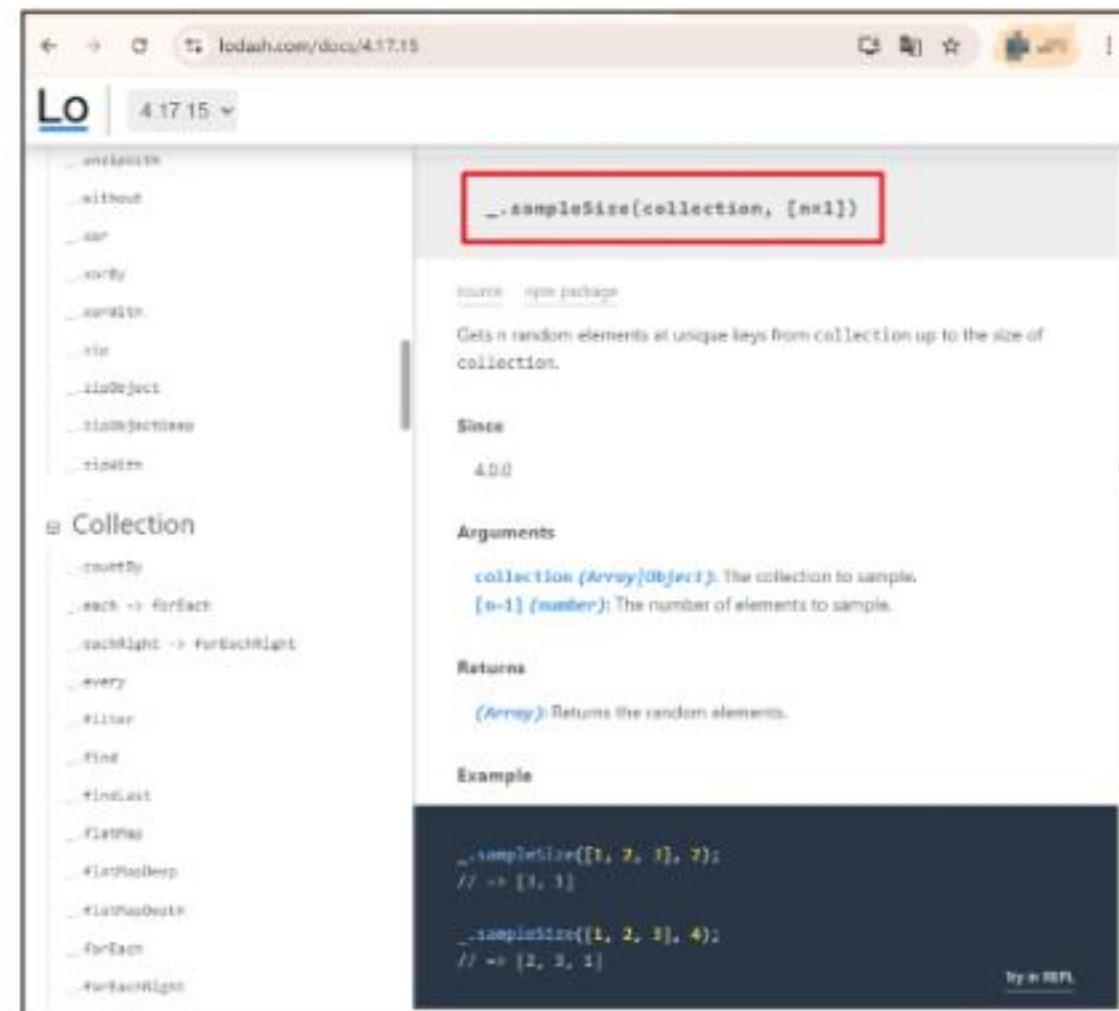
    btn.addEventListener('click', getLottery) // 3. 버튼 요소에 이벤트 핸들러를 등록
  }
</script>
```



lodash

모듈성, 성능 및 추가 기능을 제공하는 JavaScript 유틸리티 라이브러리
array/object 등 자료구조를 다룰 때, 간편하고 유용한 함수들을 제공

참고 링크: <https://lodash.com/>



<그림18 Javascript Controlling event_lodash>

이벤트 기본 동작 취소하기

이벤트 기본 동작 취소하기

- HTML의 각 요소가 기본적으로 가지고 있는 이벤트가 때로는 방해가 되는 경우가 있어 이벤트의 기본 동작을 취소할 필요가 있음
- 예시
 - form 요소의 제출 이벤트를 취소하여 페이지 새로고침을 막을 수 있음
 - <a> 태그를 클릭했을 때 페이지 이동을 막고, 대신 다른 로직을 수행하게 할 수 있음

 preventDefault

.preventDefault()

해당 이벤트에 대한 기본 동작을 실행하지 않도록 지정

preventDefault는 '회원가입 시 실수'를 막는 것과 같습니다.

회원가입 정보를 입력하다 실수로 뒤로가기를 눌렀을 때, “정말 이동하시겠습니까?”라는 문구를 본 적이 있나요?

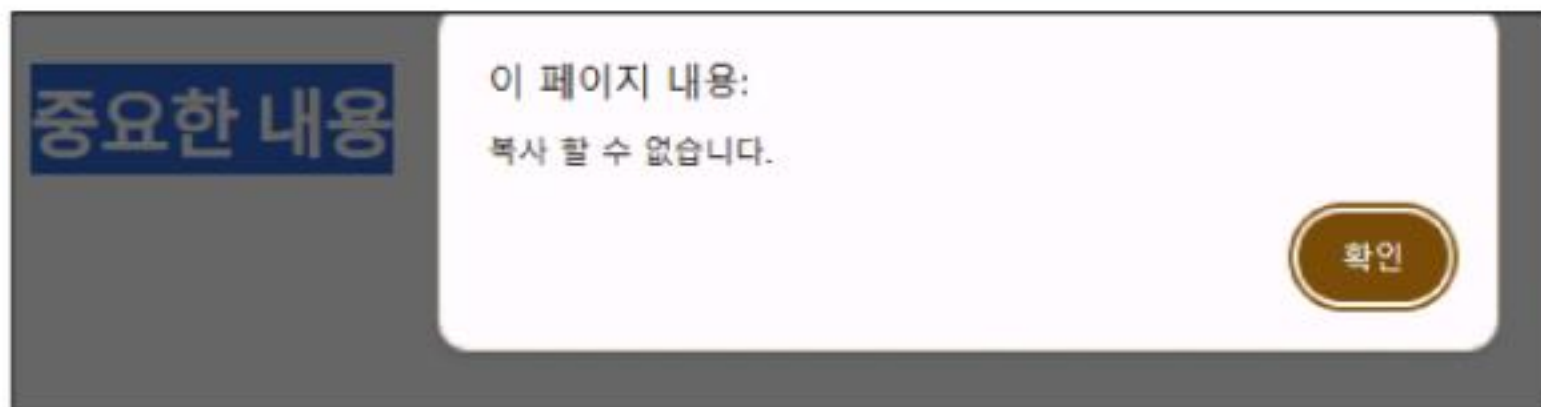
입력한 정보가 초기화되는 실수를 막아주기 위해 사용한 기능이 preventDefault 입니다.

이벤트 동작 취소 실습 목록

1. copy 이벤트 동작 취소
2. form 제출 시 페이지 새로고침 동작 취소

1. copy 이벤트 동작 취소

- 복사 이벤트를 실행하지 못하도록 하기
(콘텐츠를 복사 하는 것을 방지)



<그림19_Javascript_Controlling event_copy 이벤트 동작 취소>

```
<!-- prevent-event.html -->
```

```
<h1>중요한 내용</h1>
```

```
<script>
```

```
const h1Tag = document.querySelector('h1')
```

```
h1Tag.addEventListener('copy', function (event) {
```

```
  console.log(event)
```

```
  event.preventDefault()
```

```
  alert('복사 할 수 없습니다.')
```

```
})
```

```
</script>
```


2. form 제출 시 페이지 새로고침 동작 취소

- form 요소의 submit 동작(action 값으로 요청)을 취소 시킴
- 실습 시 submit 버튼을 눌러도 새로고침이 안되는 것을 확인할 수 있음

```
<form id="my-form">
  <input type="text" name="username">
  <button type="submit">Submit</button>
</form>
```

※ form 제출 시 기본적으로 새로고침 실행

```
const formTag = document.querySelector('#my-form')
const handleSubmit = function (event) {
  event.preventDefault()
}

formTag.addEventListener('submit', handleSubmit)
```

참고

addEventListener와 화살표 함수 관계

addEventListener에서의 화살표 함수 주의사항

- 화살표 함수는 자신만의 this를 가지지 않음
- 대신, 자신이 선언된 상위 스코프의 this를 그대로 물려받아 사용
- 따라서 이벤트 핸들러로 화살표 함수를 사용하면, this는 대부분 전역 객체(window)를 가리키게 됨
- 해결책
 - 일반 함수로 사용하기
 - 화살표 함수일 경우 event.currentTarget을 사용하기



화살표 함수

function 키워드 대신 =>를 사용해 함수를 간결하게 표현하는 문법.

<개념6_Javascript_Controlling event_화살표 함수>

```
element.addEventListener('click', function () {  
  console.log(this) // <button id="function">function</button>  
})  
  
element.addEventListener('click', () => {  
  console.log(this) // window  
})
```


이벤트

- 1917. input Event - Bad Word Filter
- 1923. Click Event - ThemeSwitcher
- 1919. change Event - Form Validator
- 1920. submit Event - Card Maker
- 3083. 이벤트 핸들러와 DOM 조작 - 기초

버블링

- 3082. 버블링과 캡처링

Q 각 문제에 올바른 답안을 생각해봅시다

1 특정 이벤트가 발생했을 때 실행되는 함수는?

- a) 이벤트 리스너
- b) 이벤트 객체
- c) 이벤트 핸들러
- d) 이벤트 타입

2 DOM 요소에 이벤트 핸들러를 등록하는 메서드는?

- a) setEvent()
- b) handleEvent()
- c) onEvent()
- d) addEventListener()

3 한 요소의 이벤트가 부모 요소로 전파되는 현상은?

- a) 캡처링 (Capturing)
- b) 버블링 (Bubbling)
- c) 위임 (Delegation)
- d) 호이스팅 (Hoisting)

4 이벤트 핸들러에 자동으로 전달되는 첫 번째 인자는?

- a) this
- b) event 객체
- c) target 요소
- d) currentTarget 요소

Q 각 문제에 올바른 답안을 생각해봅시다

5 event.target이 가리키는 대상은?

- a) 이벤트가 처음 발생한 요소
- b) 이벤트 핸들러가 연결된 요소
- c) 부모 요소
- d) 최상위 document 객체

6 event.currentTarget이 가리키는 대상은?

- a) 이벤트가 처음 발생한 요소
- b) 이벤트 핸들러가 연결된 요소
- c) 자식 요소
- d) 최상위 window 객체

7 요소의 기본 동작을 막는 메서드는?

- a) stopAction()
- b) cancelEvent()
- c) preventDefault()
- d) stopPropagation()

8 버블링의 이점을 활용하는 주요 패턴은 무엇인가요?

- a) 여러 요소에 각각 핸들러 등록
- b) 상위 요소에 핸들러 하나만 등록
- c) 모든 요소에 preventDefault() 사용
- d) 캡처링 단계에서 이벤트 처리

Q 각 문제에 올바른 답안을 생각해봅시다

9 버블링과 캡처링의 차이를 올바르게 설명한 것은?

- a) 캡처링은 하향 전파, 버블링은 상향 전파
- b) 캡처링은 상향 전파, 버블링은 하향 전파
- c) 버블링은 기본 동작을, 캡처링은 전파를 막음
- d) 두 용어는 동일하며 실제 동작에 차이가 없음

10 Event.target 과 event.currentTarget의 차이를 올바르게 설명한 것은?

- a) target은 리스너 부착 요소, currentTarget은 이벤트 발생 요소
- b) target은 이벤트 발생 요소, currentTarget은 리스너 부착 요소
- c) target은 항상 부모 요소, currentTarget은 항상 자식 요소
- d) 두 속성은 모든 상황에서 항상 동일한 요소를 가리킴

확인 문제



정답 및 해설

1. c) 이벤트 핸들러 2. d) addEventListener() 3. b) 버블링 (Bubbling) 4. b) event 객체 5. a) 이벤트가 처음 발생한 요소
6. b) 이벤트 핸들러가 연결된 요소 7. c) preventDefault() 8. b) 상위 요소에 핸들러 하나만 등록
9. a) 캡처링은 하향 전파, 버블링은 상향 전파 10. b) target은 이벤트 발생 요소, currentTarget은 리스너 부착 요소

- 이벤트 핸들러는 특정 이벤트 발생 시 호출되도록 작성된 콜백 함수입니다.
- addEventListener()는 특정 DOM 요소에 지정된 타입의 이벤트를 처리할 함수(핸들러)를 등록합니다.
- 이벤트가 가장 안쪽 요소에서 시작해 바깥쪽 부모 요소로 거슬러 올라가며 전파되는 현상입니다.
- 이벤트 발생 시, 해당 이벤트에 대한 상세 정보를 담은 event 객체가 핸들러에 자동으로 전달됩니다.
- event.Target은 이벤트가 최초로 발생한 요소, 즉 이벤트의 실제 시작이 된 요소를 가리킵니다.

- event.currentTarget은 이벤트 리스너가 실제로 연결되어 있는 요소, 즉 this와 동일한 대상을 가리킵니다.
- event.preventDefault()는 a 태그의 링크 이동이나 form의 제출과 같은 기본 동작을 취소합니다.
- 여러 하위 요소의 이벤트를 공통 상위 요소에서 한 번에 처리하여 효율성을 높일 수 있습니다.
- 캡처링은 부모에서 자식으로 하향식, 버블링은 자식에서 부모로 상향식으로 이벤트를 전달합니다.
- Target은 실제 이벤트 발생 요소를 의미하고, currentTarget은 리스너가 달린 요소(this와 동일한 대상)을 의미합니다.

활동 정리

핵심 키워드

개념	설명	예시
이벤트 (event)	웹페이지에서 일어나는 신호 또는 사건	click, input 등
이벤트 핸들러	이벤트 발생 시 실행되는 콜백 함수	btn.addEventListener('click', func)
addEventListener	DOM 요소에 이벤트 핸들러를 등록	btn.addEventListener('click', func)
이벤트 버블링	이벤트가 부모 요소로 전파되는 현상	하위 요소 클릭 시 상위 핸들러 동작
event.target	실제 이벤트가 시작된 요소를 참조	console.log(event.target)
event.currentTarget	핸들러가 연결된 요소를 참조	console.log(event.currentTarget)
preventDefault	이벤트의 기본 동작을 취소	event.preventDefault()

<표1_Javascript_Controlling event_핵심키워드>

활동 정리

오늘 공부한 내용

요약 및 정리

• 이벤트 기본 개념

- 이벤트(event)

- 웹 페이지에서 사용자의 클릭, 키보드 입력, 마우스 움직임 등 '무언가 일어났다는 신호 또는 사건'을 의미

- 이벤트 객체(event object)

- 이벤트가 발생하면, 해당 이벤트에 대한 상세 정보(발생 위치, 마우스 좌표 등)를 담은 객체가 자동으로 생성

- 이벤트 핸들러(event handler)

- 특정 이벤트가 발생했을 때 실행되도록 등록된 콜백 함수

- `addEventListener(type, handler)`

- DOM 요소에 특정 이벤트(type)가 발생했을 때 실행할 이벤트 핸들러(handler)를 등록하는 메서드

활동 정리

오늘 공부한 내용 요약 및 정리

• 이벤트 버블링 (Event Bubbling)

- 한 요소에 이벤트가 발생하면, 그 요소의 핸들러가 동작한 후 부모 요소의 핸들러, 그 부모의 부모 핸들러 순으로 최상단까지 이벤트가 전파되는 현상
- target vs currentTarget
 - 이벤트 객체 내에서 두 속성은 이벤트 전파 과정 중 요소를 참조하는 데 사용
 - event.target: 이벤트가 처음 발생한 가장 안쪽의 요소이며, 버블링이 진행돼도 이 값은 변하지 않음
 - event.currentTarget: 이벤트 핸들러가 연결된 현재 요소를 가리키고, 일반 함수 핸들러 내에서 this와 같은 값을 가짐
- 버블링의 이런 특징을 이용하면 여러 자식 요소의 이벤트를 부모 요소 하나에서 관리할 수 있음

활동 정리

웹 페이지를 하나의 '카페'라고 상상해봅시다. 손님's 주문은 어떻게 처리될까요?

1. 손님이 키오스크에서 하는 '주문'(클릭, 입력 등)이 바로 **이벤트**입니다.
2. 이 주문을 받는 키오스크가 **이벤트 리스너**입니다.
3. 주문에 맞춰 커피를 만드는 바리스타의 행동이 **이벤트 핸들러**입니다.

```
// '주문'을 받을 키오스크(버튼) 선택
const button = document.querySelector('#order-btn');

// '바리스타'의 행동(핸들러) 정의
const makeCoffee = function(event) {
  console.log('커피 제조', event); // event 객체를 통해 주문 정보 확인 가능
};

// 키오스크에 '주문'이 들어오면 '바리스타'가 행동하도록 연결
button.addEventListener('click', makeCoffee);
```

감사합니다

